



Università degli Studi dell'Insubria

Dipartimento di Scienze Teoriche e Applicate

Corso di laurea triennale in Informatica

Tesi di Laurea

Progettazione e sviluppo di un market su rete basata su microservizi tramite la piattaforma Kubernetes

Relatore

Ch.ma Prof.ssa Sabrina Sicari

Co-relatore

Prof.ssa Alessandra Rizzardi

Tesi di Laurea di

Federico Garegnani

Matricola 746789

Anno accademico

2022–2023

INDICE

1	Introduzione	3	
2	Stato dell'arte	5	
2.1	Architettura monolitica	5	
2.2	Deployment tradizionale	5	
2.3	Deployment su macchine virtuali	6	
2.4	Deployment su container	7	
3	Architettura del sistema	8	
3.1	Struttura	8	
3.2	Tecnologie utilizzate	8	
3.2.1	Python	8	
3.2.2	PostgreSQL	9	
3.2.3	Docker	9	
3.2.4	Kubernetes	9	
3.2.5	API REST	11	
3.2.6	JWT	11	
4	Implementazione	12	
5	Risultati ottenuti	13	
6	Conclusioni e sviluppi futuri	14	
	Bibliografia	16	

INTRODUZIONE

Lo sviluppo di applicazioni distribuite rappresenta una disciplina fondamentale nell'evoluzione del mondo dell'informatica. Questa pratica si concentra sulla progettazione ed implementazione di sistemi software che operano su reti di computer interconnessi, consentendo la condivisione efficiente di risorse e dati. All'interno del panorama tecnologico odierno lo sviluppo di applicazioni distribuite risulta cruciale per soddisfare le esigenze di sistemi complessi.

In ambito di sviluppo delle applicazioni suddette si riconoscono due grandi modalità di approccio al problema: un approccio più tradizionale che vede il sistema da sviluppare come un'unica unità, ed un metodo più moderno che mira a frammentare il problema principale in unità di minori dimensioni cooperanti tra loro. Il primo approccio descritto segue un'architettura monolitica e prevede che l'intera applicazione sia sviluppata e distribuita come un unico componente; contrapposta a questa, negli ultimi anni è emersa l'architettura a microservizi, un approccio architetturale che mira a scomporre la logica applicativa in componenti piccole ed autonome, consentendo lo sviluppo ed il rilascio indipendente dei singoli servizi[1].

L'approccio a microservizi porta con sé una serie di vantaggi:

SCALABILITÀ Possibilità di scalare singoli servizi in base alle esigenze, senza influenzare l'intera applicazione.

INDIPENDENZA I diversi team possono lavorare in modo autonomo su diversi microservizi.

ADATTABILITÀ È possibile sfruttare tecnologie differenti per ciascun microservizio, adattandosi meglio a requisiti o compiti specifici.

MANUTENIBILITÀ Un problema ad un microservizio non si propaga all'intera applicazione ma rimane isolato all'interno di esso, facilitandone l'individuazione.

Obiettivo di questo progetto è di sviluppare un semplice sistema gestionale di un supermercato secondo i due approcci sopra descritti e confrontarne infine le prestazioni.

L'applicazione prevede l'interazione con i clienti che acquistano prodotti dal supermercato e, per ogni loro acquisto, implementa la gestione dell'ordine: dalla rimozione dei prodotti dal magazzino fino all'avvenuto pagamento. Possiamo vedere il servizio complessivamente offerto come un e-commerce di un supermercato.

Sono definite due categorie di utenti per il servizio:

- *Utente standard.* È interessato all'acquisto dei prodotti e genera nuovi ordini.

- *Utente fornitore*. Si occupa di rifornire periodicamente il magazzino.

Gli ordini seguono questo percorso:

- a. *Creato*. L'ordine è stato ricevuto dal sistema;
- b. *Confermato*. È stato confermato che nel magazzino ogni prodotto è presente in quantità sufficiente per soddisfare l'ordine;
- c. *Pagato*. Il pagamento dell'ordine è andato a buon fine;
- d. *Spedito*. L'ordine è stato spedito ed è possibile verificare lo stato della spedizione.

Nei capitoli seguenti verranno descritti dapprima lo stile architetturale monolitico e a microservizi, li si confronterà e si elencheranno pro e contro, per poi concentrarsi nello specifico sull'applicazione sviluppata.

In dettaglio:

- *Capitolo 2*: stato dell'arte nello sviluppo di applicazioni distribuite, relativi vantaggi e svantaggi di ogni approccio.
- *Capitolo 3*: architettura del sistema e tecnologie impiegate.
- *Capitolo 4*: spiegazione delle due soluzioni proposte.
- *Capitolo 5*: confronto di performance tra le due soluzioni.
- *Capitolo 6*: conclusioni e possibili sviluppi futuri.

STATO DELL'ARTE

In ambito di sviluppo e deployment di applicazioni distribuite si distinguono diversi metodi; descriveremo nel corso di questo capitolo pro e contro di ogni approccio.

2.1 ARCHITETTURA MONOLITICA

L'architettura monolitica è il modello tradizionale di sviluppo di un'applicazione in cui l'intera logica applicativa è implementata in un unico componente. Un simile approccio è ideale per applicazioni di dimensioni ridotte e che non devono soddisfare un elevato numero di richieste, presentando infatti una minore complessità di sviluppo ed offrendo performance migliori[8]. All'aumentare della dimensioni dell'applicazione o del traffico da soddisfare il modello monolitico inizia però a mostrare i propri limiti:

- *Il sistema non è scalabile*: Se una singola parte dell'applicazione è soggetta a un carico di lavoro elevato, questa non può essere ridimensionata individualmente, ma si è obbligati a scalare l'intera applicazione.
- *Poca flessibilità*: Tutti i moduli devono obbligatoriamente utilizzare la stessa tecnologia, una modifica in questo senso comporterebbe la riscrittura di buona parte dell'applicazione. Alcuni vincoli potrebbero essere: la scelta di un linguaggio di programmazione unico, di un framework specifico, di un unico sistema per la gestione del codice sorgente o di un'unica piattaforma di hosting.
- *Poca resilienza*: Un errore all'interno di un modulo può portare al blocco dell'intera applicazione.
- *Mantenimento*: Anche una singola modifica all'interno di un modulo costringe ad eseguire nuovamente il deployment dell'intera applicazione.

2.2 DEPLOYMENT TRADIZIONALE

La metodologia tradizionale di deployment prevede l'esecuzione delle applicazioni direttamente sul server fisico. Sebbene semplice ed immediato questo modo di procedere non è ottimale, non permette infatti di scalare l'applicazione ponendo dei limiti sull'utilizzo di risorse (CPU, RAM, disco, ...) e si corre il rischio che una seconda applicazione non riesca a garantire le prestazioni desiderate. Si potrebbe cercare un'alternativa a questo problema eseguendo ogni applicazione su un server fisico distinto, ma anche

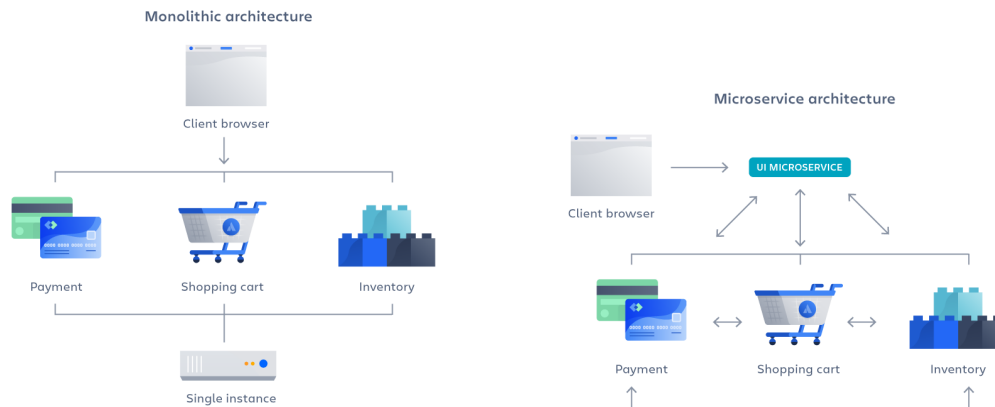


Figura 2.1: Principio di funzionamento di un'applicazione monolitica e una a microservizi[8]

una tale soluzione non è adatta a un contesto reale risultando molto costosa e portando a un sottoutilizzo delle risorse complessive[1].

2.3 DEPLOYMENT SU MACCHINE VIRTUALI

Le macchine virtuali sono pacchetti software piuttosto pesanti, in grado di eseguire l'emulazione dei dispositivi hardware di basso livello quali CPU, RAM, dischi e schede di rete. Il deployment su macchine virtuali (anche chiamato deployment virtualizzato) prevede l'esecuzione di più macchine virtuali sul server fisico, ciascuna delle quali esegue un sistema operativo che a sua volta esegue una o più applicazioni. Un simile approccio è in grado di far fronte ad alcune delle problematiche del deployment tradizionale:

- *Limitazione delle risorse:* In funzione di quanto detto sopra è possibile limitare le risorse dedicate ad una macchina virtuale in termini di tempo di CPU, quantità di RAM, etc.
- *Isolamento:* Le macchine virtuali vengono eseguite in un ambiente isolato e completamente autonomo che le rende immuni da tentativi di interferenza da parte di altre macchine virtuali attive sullo stesso host.

Contrariamente, i principali svantaggi delle macchine virtuali riguardano la ridotta velocità di generazione e distribuzione delle stesse e il maggiore spazio di archiviazione richiesto: trattandosi di *immagini* di un sistema indipendente possono infatti richiedere diverso tempo per essere generate e configurate, l'affiancamento di diverse macchine virtuali porta inoltre alla duplicazione del sistema operativo installato su di esse, con un conseguente utilizzo di spazio di archiviazione.

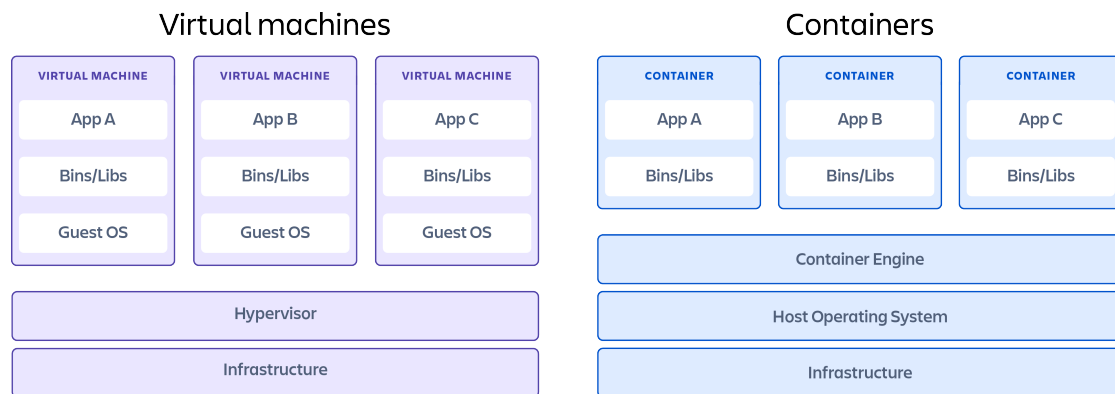


Figura 2.2: Confronto tra un sistema basato su macchine virtuali e container[9]

2.4 DEPLOYMENT SU CONTAINER

Un *container* è un modello di virtualizzazione più leggero rispetto ad una macchina virtuale che racchiude un software e tutte le dipendenze necessarie ad eseguirlo. Al pari delle macchine virtuali i container rappresentano un ambiente isolato, con risorse limitate ed un proprio file system per l'esecuzione del software ma non virtualizzano un intero sistema operativo. L'uso di container offre una grande efficienza in fase di sviluppo e di esecuzione, contengono infatti solo software di alto livello e sono molto veloci da modificare, oltre al fatto che non dovendo virtualizzare un sistema operativo la loro esecuzione risulta molto più rapida e meno esosa di risorse hardware. Per finire molti sistemi runtime per container mettono a disposizione degli sviluppatori repository pubbliche con immagini preconfezionate di software popolari, il tutto al fine di avere uno sviluppo ancora più rapido [1, 9].

ARCHITETTURA DEL SISTEMA

Nel corso di questo capitolo verrà dapprima posta l'attenzione sulla struttura dei due sistemi sviluppati, per poi focalizzarsi sulla descrizione delle tecnologie utilizzate.

3.1 STRUTTURA

Come già precedentemente evidenziato il sistema è stato sviluppato seguendo due filosofie differenti, dapprima secondo un approccio monolitico, in seguito seguendo un approccio a microservizi; le due applicazioni sono tra di loro molto simili, condividono buona parte del codice e sfruttano le stesse tecnologie.

L'applicazione sviluppata in modo monolitico espone alcune chiamate API REST, che le permettono di ricevere richieste ed inviare risposte agli applicativi lato client attraverso il protocollo HTTP. Lo scambio di informazioni tra client e server avviene scambiando messaggi in formato JSON ed è protetto con algoritmi di crittografia e metodi di autenticazione tramite token, il tutto per garantire sicurezza e privacy dei dati scambiati. Per la conservazione dei dati persistenti (quali informazioni sugli utenti, sulla merce in magazzino o sugli ordini) si utilizza infine un database PostgreSQL.

L'applicazione a microservizi nasce dall'applicazione monolitica e la scompone in un numero di unità pari al numero di servizi complessivamente offerti, nel contesto applicativo ciascun modulo si riduce così ad esporre una singola chiamata API, occupandosi di un singolo aspetto tra i passi successivi che portano dalla scelta dei prodotti fino alla spedizione dell'ordine. Il sistema si compone in questo caso di un cluster di server eseguiti all'interno di container e, in un contesto più ampio, di un cluster Kubernetes.

3.2 TECNOLOGIE UTILIZZATE

3.2.1 *Python*

Python è un linguaggio di programmazione *general-purpose* orientato agli oggetti e ad alto livello, molto diffuso in ambiti quali l'analisi dei dati, l'intelligenza artificiale, la finanza e la sicurezza informatica[7, pp. 7–8]. Pone grande enfasi sulla leggibilità e dispone di una sintassi particolarmente semplice che, insieme a una vastissima gamma di librerie, permette una grande riduzione in termini di linee di codice[11].

3.2.2 PostgreSQL

PostgreSQL è un Database Management System (DBMS) di tipo relazionale e ad oggetti, molto potente ed open source. Si tratta di una valida alternativa ad altri progetti sia liberi che con licenza ed è divenuto celebre nel tempo per alcuni suoi punti di forza quali la programmabilità, permettendo un incremento nel livello di astrazione, affidabilità e prestazioni raggiunte[12, 13].

3.2.3 Docker

Docker è una piattaforma aperta per lo sviluppo, la consegna e l'esecuzione di applicazioni all'interno di container. All'interno di Docker sono definiti essenzialmente due oggetti principali:

IMMAGINI Un immagine è un template di sola lettura contenente le istruzioni per generare un nuovo container. La generazione di nuove immagini è estremamente semplice ed avviene sulla base di istruzioni definite in un *Dockerfile*, un file contenente le istruzioni per la generazione di un ambiente preconfigurato e pronto per l'esecuzione. Ogni istruzione all'interno del Dockerfile corrisponde a un livello nella generazione dell'immagine, facilitando la condivisione e la distribuzione della stessa; quando un Dockerfile viene modificato e l'immagine è ricompilata, solo i livelli che hanno subito modifiche vengono ricompilati. Si tratta quindi di strutture molto leggere, piccole e veloci se comparate ad altri modelli di virtualizzazione[2].

CONTAINER L'idea alla base dei container è già stata spiegata nella Sezione 2.4, nel contesto di Docker i container assumono il ruolo di istanze eseguibili delle immagini[2].

Docker risulta ottimo per la gestione di container su un singolo host, offrendo strumenti per la limitazione e il monitoraggio delle risorse o il bilanciamento del carico (attraverso *Docker Swarm* o *Overlay Network Driver*), con caratteristiche che risultano tuttavia limitate rispetto ad altri orchestratori più avanzati[10].

3.2.4 Kubernetes

Kubernetes è una piattaforma open source per l'automazione della distribuzione, scalabilità e gestione di applicazioni eseguite su container, azioni che nell'insieme prendono il nome di *orchestrazione*. La necessità di orchestratori di container nasce in contesti di produzione, laddove c'è la necessità di gestire un gran numero di container ed assicurarsi che i servizi offerti siano sempre attivi[2].

Alcune delle caratteristiche principali di Kubernetes sono:

GESTIONE DELLE RISORSE Permette di porre limiti sulle risorse utilizzabili da un container (RAM, CPU, disco, etc.).

BILANCIAMENTO DEL CARICO È in grado di gestire più repliche di uno stesso container e gestire il traffico in ingresso distribuendolo uniformemente sulle istanze attive. Fornisce inoltre la funzione di *auto-scaling* che permette di variare il dinamicamente il numero di istanze di un container sulla base del traffico di rete.

SELF-HEALING Termina automaticamente i container che si sono bloccati e li sostituisce con nuove istanze degli stessi.

CONFIGURAZIONE DICHIARATIVA È possibile lavorare in modalità DSC (*Desired State Configuration*), in cui un cluster può essere configurato utilizzando dei file dichiarativi che descrivono lo stato in cui si vuole portare il sistema. La fase di transizione dallo stato attuale del cluster allo stato desiderato è del tutto trasparente all'utente e non è in alcun modo modificabile, sarà Kubernetes a prendere tutte le decisioni in merito ed evolvere il sistema.

Gli elementi chiave di Kubernetes sono *Nodi*, *Cluster*, *Pod*, *Deployment*, *Service*. Di seguito una breve presentazione per ognuno di essi:

CLUSTER Un cluster è l'unità logica più ampia definita in Kubernetes e rappresenta l'ambiente all'interno del quale sono orchestrati i container.

NODI I nodi possono essere macchine fisiche o virtuali e rappresentano i punti tra i quali è suddiviso il carico di lavoro. Ne sono definiti due tipi: i *control plane node* e i *worker node*; i primi gestiscono l'interazione con il cluster e si occupano di distribuire uniformemente il carico di lavoro tra i *worker node*; i secondi sono nodi operativi, sui quali vengono istanziati i *pod*[3].

POD Un Pod è la più piccola unità definibile con Kubernetes e corrisponde a un gruppo di uno o più container che condividono spazio di archiviazione e risorse di rete. Ogni pod modella di fatto un host logico[4].

DEPLOYMENT E REPLICASET Un *Deployment* è un oggetto Kubernetes che permette di dichiarare come le applicazioni al suo interno debbano essere gestite, distribuite e quante risorse gli si debba dedicare. All'interno del deployment viene solitamente definito un oggetto *ReplicaSet*, il quale racchiude un insieme di pod che condividono una configurazione comune. La risorsa *ReplicaSet* si occupa di mantenere il numero di repliche desiderato per ciascun pod, istanziandone di nuovi o distruggendo quelli bloccati. Un Deployment è un concetto di più alto livello che permette di gestire i ReplicaSet e fornire aggiornamenti sulla configurazione dei pod in maniera dichiarati-

va[5]; è ad esempio possibile specificare le immagini da utilizzare per la generazione di container, i numeri di porta da esporre, etc.

SERVICE Un oggetto *Service* rappresenta un'astrazione al fine di esporre un gruppo di pod sulla rete. La necessità di definire un tale oggetto nasce dalla natura effimera dei pod, che come precedentemente spiegato possono essere distrutti e rigenerati; da ciò consegue che non è possibile conoscere quali unità sono attive e funzionanti in un dato momento. La configurazione di un service contiene il nome del service, un selettore per identificare i pod raggiungibili e i numeri di porta accessibili dall'esterno, generando di fatto un'astrazione dagli indirizzi IP interni di un insieme di pod[6].

3.2.5 API REST

Per API REST (o RESTful) si intende una *Application Programming Interface* progettata secondi i principi dello stile architetturale REST (*REpresentational State Transfer*). I principi guida di REST sono definiti al fine di promuovere semplicità e scalabilità:

1. *Interfaccia uniforme*: Ogni risorsa trasferita dal server deve rispettare delle condizioni di generalità, come: identificazione univoca della risorsa; manipolazione della risorsa tramite rappresentazioni (le risorse devono essere rappresentate in un formato standard come JSON o XML); autodescrizione delle risorse.
2. *Architettura client-server*: Client e server possono essere sviluppati separatamente, in modo indipendente uno dall'altro.
3. *Stateless*: Ogni richiesta dal client al server deve contenere tutte le informazioni necessarie per comprendere e completare la richiesta, il server non deve far uso di informazioni memorizzate precedentemente.
4. *Cache*: Ogni risorsa deve specificare se può essere memorizzata e riutilizzata in seguito come risposta a una richiesta equivalente.
5. *Sistema a più livelli*: È possibile definire una struttura gerarchica tra livelli, così che all'arrivo di una richiesta il server principale può inoltrare quest'ultima ad altri server di livello inferiore. Solo il livello più esterno deve però essere visibile all'utente.
6. *Codice on demand*: Oltre a risorse statiche, all'occorrenza può essere inviato anche del codice sorgente.

3.2.6 JWT

IMPLEMENTAZIONE

RISULTATI OTTENUTI

CONCLUSIONI E SVILUPPI FUTURI

ELENCO DELLE FIGURE

- Figura 2.1 Principio di funzionamento di un'applicazione monolitica e una
a microservizi[8] 6
- Figura 2.2 Confronto tra un sistema basato su macchine virtuali e contai-
ner[9] 7

BIBLIOGRAFIA

- [1] *Overview on Kubernetes*. URL: <https://kubernetes.io/docs/concepts/overview/>.
- [2] *Docker overview*. URL: <https://docs.docker.com/get-started/overview/>.
- [3] *Kubernetes nodes*. URL: <https://kubernetes.io/docs/concepts/architecture/nodes/>.
- [4] *Pods – Pods are the smallest deployable units of computing that you can create and manage in Kubernetes*. URL: <https://kubernetes.io/docs/concepts/workloads/pods/>.
- [5] *ReplicaSet – A ReplicaSet’s purpose is to maintain a stable set of replica Pods running at any given time. As such, it is often used to guarantee the availability of a specified number of identical Pods*. URL: <https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/>.
- [6] *Service – In Kubernetes, a Service is a method for exposing a network application that is running as one or more Pods in your cluster*. URL: <https://kubernetes.io/docs/concepts/services-networking/service/#endpoints>.
- [7] Mark Lutz. *Learning Python*. O’Reilly Media, 2009. ISBN: 978-0-596-15806-4.
- [8] *Microservices vs. monolithic architecture – When monoliths grow too big it may be time to transition to microservices*. URL: <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>.
- [9] *Containers vs. virtual machines – Learn the differences between containers and virtual machines, popular providers for each, and how they can be used together*. URL: <https://www.atlassian.com/microservices/cloud-computing/containers-vs-vms>.
- [10] *Kubernetes vs Docker: A comprehensive comparison*. URL: <https://www.civo.com/blog/kubernetes-vs-docker-a-comprehensive-comparison#:~:text=Improved%20container%20orchestration%3A%20Kubernetes%20provides,demand%2C%20ensuring%20efficient%20resource%20utilization..>
- [11] *Python (programming language)*. URL: [https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language)).
- [12] *PostgreSQL – About*. URL: <https://www.postgresql.org/about/>.
- [13] *PostgreSQL*. URL: <https://it.wikipedia.org/wiki/PostgreSQL>.